

Literature Review

An Automotive Manufacturing Knowledge Graph for Equipment Diagnostics using Neo4j

Vellore Institute of Technology (VIT) — Postgraduate DBMS Project

1. Introduction

This review surveys the published work most relevant to the design choices behind the proposed Automotive Manufacturing Knowledge Graph: (a) how graph databases such as Neo4j perform relative to relational database management systems (RDBMS), particularly on the kind of multi-hop, relationship-heavy queries this project relies on; and (b) how existing research constructs knowledge graphs for equipment fault diagnosis in industrial and engineering domains. The goal is to ground the project's core design decision — modelling machines, error codes, repair procedures, spare parts and warehouse locations as a Neo4j property graph rather than a set of relational tables — in existing evidence, while being candid about where that evidence is mixed.

2. Graph Database versus Relational Database Performance

The foundational reference for the property graph model used throughout this project is Robinson, Webber and Eifrem's *Graph Databases*, which formalises the idea of index-free adjacency: because each node stores direct references to its adjacent nodes, traversing a relationship is a constant-time pointer lookup rather than a join computed at query time. This is the theoretical basis for the project's claim that tracing an error code through to its repair procedure and spare part should scale better in a graph model than in an RDBMS as the schema grows more interconnected.

Empirical work complicates a simple “graphs are always faster” narrative, however. Kotiranta, Junkkari and Nummenmaa (2022) built a purpose-made invoicing-database test bench to compare Neo4j against MySQL and MariaDB across query tasks of increasing complexity, including recursive traversals. For simple to moderately complex lookups, Neo4j was consistently around three times faster than the relational alternatives. But as query complexity increased — particularly for aggregation queries built around a defined key, and for long recursive chains once indexing was applied — modern relational engines caught up and in several cases dramatically overtook Neo4j, by factors of tens to over a thousand. Their conclusion is that no database category is universally superior; the right choice depends on the specific balance of query complexity, indexing strategy and data volume in the target application.

Sandell, Asplund, Ayele and Duneld (2024) offer a more favourable data point for graph databases, but under different conditions. Benchmarking ArangoDB, MySQL and Neo4j on a multi-entity biomedical connected-data set using a formal ANOVA-based statistical design, they found Neo4j's query time was significantly faster than both alternatives, and that it also used less memory than MySQL, with no significant differences in CPU or energy usage across the three systems. Their workload — multi-hop pattern matching across several distinct node types — is structurally closer to this project's diagnostic lookups (machine → error code → procedure → part → bin) than the aggregation-heavy invoicing workload used by Kotiranta et al., which is a useful point to make when justifying Neo4j for this specific use case rather than as a blanket claim. It should be noted, however, that this source is an arXiv preprint rather than a peer-reviewed publication, and its conclusions should therefore be weighted accordingly.

A useful third data point comes from Monteiro, Sá and Bernardino (2023), who evaluated the four highest-ranked graph databases — JanusGraph, Nebula Graph, Neo4j and TigerGraph — using the Linked Data Benchmark Council's Social Network Benchmark across five scale factors from 112 MB to 12.2 GB. Neo4j returned the best result on every metric measured: lowest average node load time, lowest total query execution time (24.30 minutes against 60.97 for TigerGraph and 101.58 for Nebula Graph), lowest average CPU usage at 26%, and lowest average RAM usage at 42%. This does not speak to the graph-versus-relational question directly, but it does address a separate design decision this project must justify: given that a graph database is appropriate for the workload, Neo4j specifically is a defensible choice on measured performance grounds rather than merely on familiarity or documentation quality.

Taken together, these three studies support a narrower and more defensible claim than “Neo4j is faster.” The evidence indicates that graph databases win on short, relationship-dense traversals and lose on large-scale, key-based aggregation; that Neo4j is the strongest performer within the graph-database category; and that the workload profile, not the database category, should drive the decision. Since the diagnostic queries this project depends on fall squarely into the first pattern, the workload-fit argument is the one this project should make.

3. Knowledge Graph Construction for Equipment Fault Diagnosis

Several recent studies build fault-diagnosis knowledge graphs that follow a similar pattern to the one proposed here. The most comprehensive overview is Wan et al. (2024), a review in the Journal of Manufacturing Systems mapping research topics, applications and open problems for knowledge graphs in smart manufacturing. It provides the framing this project needs in order to position itself within an established research area rather than presenting the approach as novel in itself, and it identifies which parts of the pipeline — schema design, extraction, fusion, reasoning — are comparatively mature and which remain open.

At the level of individual systems, Tang, Chi, Cui, Ip, Yung and Xie (2023) construct a fault knowledge graph for aircraft maintenance: they first define a schema layer covering the knowledge elements needed for diagnosis, then use deep learning together with heuristic rules to extract fault knowledge from a mix of structured and unstructured maintenance data, and finally layer a question-answering system on top so maintenance engineers can query the graph directly rather than searching manuals by hand. The overall shape of their pipeline — schema design, extraction from heterogeneous documentation, then a query-facing layer for engineers — maps closely onto this project's own plan for linking machine manuals, SOPs and error logs.

Closer still to a production-line setting, Chen, Zhou, Zhang and Zha (2025) build a fault knowledge graph for intelligent production lines, defining an ontology whose entity types — fault subject, fault content, fault cause, detection method and maintenance solution — correspond almost directly to the machine, error code and repair procedure entities in this project's proposed schema. They extract entities and relations from maintenance texts using an ALBERT-BiLSTM-Attention-CRF model, reporting precision, recall and F1 of 92.57%, 94.35% and 93.45% respectively, and store the resulting triples in Neo4j for visualisation and query. Their CNC-machining-centre case study is the closest published analogue to this project's assembly-line scope, and their ontology is a reasonable starting point to adapt rather than design from scratch.

Han, Wang, Wang and Chen (2022) address a dimension that static schema design tends to overlook: how a fault diagnosis knowledge graph should evolve as new fault cases accumulate. Their work on construction and evolution of fault knowledge graphs in industrial process settings is relevant to this project's longer-term viability, since an assembly-line graph populated once from current manuals will drift out of date as equipment is replaced and new failure modes appear. Cai, Jiang, Wu, Wang, Liu and Song (2024) take a complementary angle, proposing a multi-level knowledge graph for intelligent manufacturing and combining it with Bayesian reasoning over relationship paths to trace fault sources — demonstrating that the graph can serve as an inference substrate rather than only as a lookup structure.

Two studies point toward where such a graph can go beyond retrieval. Mao, Wang, Jiang, Xu and Guo (2024) apply a graph convolutional neural network over a fault knowledge graph to perform machine fault diagnosis, and Xia, Liang, Leng and Zheng (2023) combine a knowledge graph with a graph neural network to recommend maintenance plans for complex industrial equipment. Neither is required for this project's core deliverable, which is a queryable diagnostic and inventory graph rather than a predictive system. They are cited here because they establish that the schema this project builds is a foundation on which learned reasoning can later be layered — which is a more credible framing of future work than an unsupported claim that the graph “could support machine learning.”

Huang and Song (2025) tackle the extraction step specifically, proposing a pipeline that uses a large language model, guided by prompt engineering and domain ontology constraints, to automatically pull structured knowledge out of maintenance documents and operating manuals. Semantic matching and conflict detection are applied during knowledge fusion to keep the resulting graph internally consistent, after which the extracted knowledge triples are loaded into Neo4j using Cypher statements. Their results show this reduces manual extraction effort considerably. This is directly relevant to the data-ingestion stage of the present project: manually converting PDFs, SOPs and error-code tables into Cypher CREATE statements is likely to be one of the more time-consuming parts of implementation, and an LLM-assisted extraction step along these lines is a reasonable direction for future extension.

4. Industrial Framing: Neo4j in Manufacturing

Beyond the academic literature, Neo4j's own manufacturing use-case material frames graph databases as a way to surface hidden links across production data, with stated applications in streamlining supply chains, advancing predictive maintenance and strengthening quality control at scale. This is a practitioner-facing source rather than a peer-reviewed one, so it should be cited alongside — not instead of — the academic sources above, but it is useful corroboration that the downtime-reduction motivation behind this project (linking an error code directly to its repair procedure, part and warehouse bin to cut mean time to repair) matches how the technology is actually being pitched and adopted in industry.

5. Synthesis and Positioning of the Present Project

Taken together, the literature suggests two things for this project. First, the performance case for Neo4j is strongest precisely for the kind of query this project needs — short, multi-hop traversals across a handful of connected entity types — and weaker for the kind of large-scale aggregation queries this project does not need, which means the choice of Neo4j should be justified on workload fit

rather than a general performance claim. Within the graph-database category, the benchmark evidence independently supports Neo4j over its closest competitors.

Second, while fault-diagnosis knowledge graphs have been built for aircraft maintenance, industrial process plant, CNC machining centres and intelligent production lines, and while the schema patterns and extraction techniques are by now well established, none of the sources reviewed combine an automotive assembly-line equipment graph with an integrated spare-parts and warehouse-location layer in a single Neo4j schema in the way this project proposes. The reviewed diagnostic graphs terminate at the maintenance solution; they identify what is wrong and what should be done, but not which part is required or where in the warehouse it physically sits. That combination — diagnostic knowledge graph plus inventory and location data in one connected model — is the specific gap this project aims to address, applying the schema-design and extraction lessons from the fault-diagnosis literature to a domain and data combination not yet covered in the sources surveyed here.

Two limitations of this review should be stated plainly. The scope is deliberately narrow: it covers graph-versus-relational performance and fault-diagnosis knowledge graph construction, and does not survey the adjacent literature on ontology engineering standards, spare-parts inventory optimisation, or maintenance scheduling, any of which would bear on a fuller treatment of the problem. And the claim of novelty rests on the absence of a matching study within the sources surveyed, which is weaker than a systematic search would provide; it should be read as a gap identified in this review rather than an established gap in the field.

Research Papers

Peer-reviewed and scholarly sources cited in this review (11 items), listed alphabetically.

Cai, C., Jiang, Z., Wu, H., Wang, J., Liu, J., & Song, L. (2024). Research on knowledge graph-driven equipment fault diagnosis method for intelligent manufacturing. *The International Journal of Advanced Manufacturing Technology*, 130(9), 4649–4662.

Link: <https://doi.org/10.1007/s00170-024-12998-x>

Chen, Y., Zhou, M., Zhang, M., & Zha, M. (2025). Knowledge-graph-driven fault diagnosis methods for intelligent production lines. *Sensors*, 25(13), 3912.

Link: <https://doi.org/10.3390/s25133912>

Han, H., Wang, J., Wang, X., & Chen, S. (2022). Construction and evolution of fault diagnosis knowledge graph in industrial process. *IEEE Transactions on Instrumentation and Measurement*, 71, 1–12.

Link: <https://doi.org/10.1109/TIM.2022.3200429>

Huang, Y., & Song, X. (2025). A method for constructing equipment fault diagnosis knowledge graph based on large language model. In *Intelligent Simulation (CSC 2025)*, Communications in Computer and Information Science, vol. 2680 (pp. 203–217). Springer, Singapore.

Link: https://doi.org/10.1007/978-981-95-2748-9_13

Kotiranta, P., Junkkari, M., & Nummenmaa, J. (2022). Performance of graph and relational databases in complex queries. *Applied Sciences*, 12(13), 6490.

Link: <https://doi.org/10.3390/app12136490>

Mao, Z., Wang, H., Jiang, B., Xu, J., & Guo, H. (2024). Graph convolutional neural network for intelligent fault diagnosis of machines via knowledge graph. *IEEE Transactions on Industrial Informatics*, 20(5), 7862–7870.

Link: <https://doi.org/10.1109/TII.2023.3341931>

- Monteiro, J., Sá, F., & Bernardino, J. (2023). Experimental evaluation of graph databases: JanusGraph, Nebula Graph, Neo4j, and TigerGraph. *Applied Sciences*, 13(9), 5770.
Link: <https://doi.org/10.3390/app13095770>
- Sandell, J., Asplund, E., Ayele, W. Y., & Duneld, M. (2024). Performance comparison analysis of ArangoDB, MySQL, and Neo4j: An experimental study of querying connected data. *arXiv preprint arXiv:2401.17482*.
Link: <https://arxiv.org/abs/2401.17482>
- Tang, X., Chi, G., Cui, L., Ip, A. W. H., Yung, K. L., & Xie, X. (2023). Exploring research on the construction and application of knowledge graphs for aircraft fault diagnosis. *Sensors*, 23(11), 5295.
Link: <https://doi.org/10.3390/s23115295>
- Wan, Y., Liu, Y., Chen, Z., Chen, C., Li, X., Hu, F., & Packianather, M. (2024). Making knowledge graphs work for smart manufacturing: Research topics, applications and prospects. *Journal of Manufacturing Systems*, 76, 103–132.
Link: <https://doi.org/10.1016/j.jmsy.2024.07.009>
- Xia, L., Liang, Y., Leng, J., & Zheng, P. (2023). Maintenance planning recommendation of complex industrial equipment based on knowledge graph and graph neural network. *Reliability Engineering & System Safety*, 232, 109068.
Link: <https://doi.org/10.1016/j.ress.2022.109068>

References

Books, practitioner documentation, and other non-peer-reviewed sources (2 items), listed alphabetically.

- Neo4j, Inc. (n.d.). Manufacturing — Neo4j industry use cases. Retrieved August 2026.
Link: <https://neo4j.com/developer/industry-use-cases/manufacturing/>
- Robinson, I., Webber, J., & Eifrem, E. (2015). *Graph databases: New opportunities for connected data* (2nd ed.). O'Reilly Media.